

# **FLUTTER**

# **PRODUCTION ARCHITECTURE STANDARD**

Clean Architecture + Repository Pattern + Riverpod + GoRouter

For AI-Assisted Architecture Generation

Reference Document for AI Agents

# TABLE OF CONTENTS

1. Purpose & Scope
2. AI Role & Responsibilities
3. Architecture Principles
4. Project Constraints (V1)
5. Dependency Rules
6. Project Folder Structure
7. Feature-First Architecture
8. Data Layer Design
9. Domain Layer Design
10. Presentation Layer Design
11. Routing & Navigation
12. State Management (Riverpod)
13. Database & Persistence
14. Error Handling
15. Testing Standards
16. Code Quality & Naming
17. AI Development Process (13 Phases)
18. Do NOT Over-Engineer
19. Future Scalability
20. Final Architecture Checklist

## 1. PURPOSE & SCOPE

This document defines the mandatory engineering standard for building Flutter applications using Clean Architecture.

**This document defines HOW the application must be designed and built.**

The PROJECT DESCRIPTION defines WHAT the application must do.

**The AI must analyze BOTH before writing implementation code.**

## 2. AI ROLE & RESPONSIBILITIES

Act as a **Senior Flutter Software Architect and Production Engineer**.

### You are NOT building:

- Tutorial code • Prototype • Proof-of-concept • Throwaway demo

### You ARE building:

Maintainable production-quality Flutter application that another developer can continue developing later.

### Your Responsibilities:

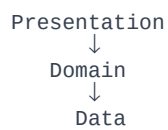
- Understand project requirements
- Identify actual features
- Design appropriate architecture
- Design data model & schema
- Design local persistence
- Design navigation structure
- Design state management
- Create scalable folder structure
- Implement features incrementally
- Write maintainable, testable code
- Handle errors & edge cases
- Document architecture decisions

### 3. ARCHITECTURE PRINCIPLES

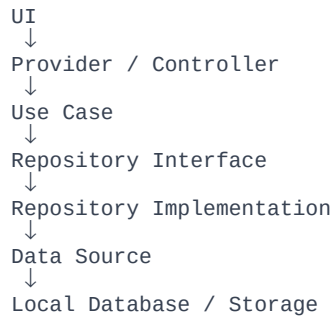
#### Use Feature-First Clean Architecture with:

- Repository Pattern
- Dependency Inversion
- Separation of Concerns
- Riverpod (state management)
- GoRouter (navigation)
- Local persistence (offline-first)

#### Primary Dependency Direction:



#### More Specifically:



**Critical Rule: The UI must NEVER directly access infrastructure.**

## 4. PROJECT CONSTRAINTS (V1)

**Unless the developer explicitly says otherwise, V1 application MUST:**

- ✓ Use Flutter & Dart
- ✓ Work without a backend
- ✓ Work without server database
- ✓ Store application data locally
- ✓ Prefer offline-first functionality
- ✓ Be usable without internet wherever possible
- ✓ Not require an admin panel
- ✓ Not require paid APIs
- ✓ Not require paid cloud services

**DO NOT:**

- ✗ Add a backend simply because it could be useful in future
- ✗ Add admin panel
- ✗ Introduce paid APIs
- ✗ Introduce unnecessary cloud infrastructure
- ✗ Implement features for hypothetical V2
- ✗ Over-engineer with enterprise patterns

## 5. DEPENDENCY RULES (MANDATORY)

The dependency direction must be respected:

✓ **CORRECT:**

Presentation → Domain → Data

✗ **NOT ALLOWED (Direct access to infrastructure):**

UI → Database

UI → SharedPreferences

UI → SQLite

UI → File System

UI → Data Source

UI → Repository Implementation

**INSTEAD use layers:**

```
UI
↓
Provider
↓
Use Case
↓
Repository
↓
Data Source
```

## 6. PROJECT FOLDER STRUCTURE

### Reference Architecture:

```
project_name/  
  android/  
  ios/  
  web/  
  assets/  
    images/  
    icons/  
    fonts/  
    animations/  
  lib/  
    main.dart  
    app/  
      app.dart  
      router/  
      theme/  
      config/  
    core/  
      constants/  
      errors/  
      database/  
      storage/  
      services/  
      utils/  
      widgets/  
    features/  
    shared/  
  test/  
  integration_test/  
  docs/  
    architecture.md  
    requirements.md  
    decisions.md  
  pubspec.yaml  
  analysis_options.yaml  
  .gitignore
```

**IMPORTANT: This is a REFERENCE architecture. Do NOT create unnecessary folders. Only create folders required by the actual project.**

## 7. FEATURE-FIRST ARCHITECTURE

Major application functionality belongs inside: `lib/features/`

Example structure for a vehicle tracking app:

```
features/  
  ■■■ vehicles/  
  ■■■ fuel/  
  ■■■ maintenance/  
  ■■■ expenses/  
  ■■■ documents/
```

Example structure for a PDF application:

```
features/  
  ■■■ pdf_viewer/  
  ■■■ pdf_merge/  
  ■■■ pdf_split/  
  ■■■ pdf_compress/  
  ■■■ pdf_history/
```

Example structure for a finance application:

```
features/  
  ■■■ accounts/  
  ■■■ transactions/  
  ■■■ categories/  
  ■■■ reports/  
  ■■■ settings/
```

**Critical Rule: Do NOT blindly copy example architecture. Analyze the project first, then create project-specific features based on actual requirements.**



## 8. FEATURE STRUCTURE

Every major feature follows this structure:

```
feature_name/  
  data/  
    datasources/  
    models/  
    repositories/  
  domain/  
    entities/  
    repositories/  
    usecases/  
  presentation/  
    providers/  
    screens/  
    widgets/
```

**Important:** Do not create every folder if the feature does not require it. Only create necessary folders.

## 9. DATA LAYER DESIGN

The data layer handles data access and persistence.

### Data Sources:

Responsible for actual data access. Examples:

- Local Database (SQLite, Isar, Drift)
- Local File Storage
- Device Storage
- Device APIs
- Future: Remote API

### Models:

Models represent stored/transferred data. They handle:

- Serialization
- Deserialization
- Database mapping
- JSON conversion

### Repository Implementations:

Repository implementations connect domain contracts with data sources.

Example:

```
note_repository_impl.dart  
vehicle_repository_impl.dart  
transaction_repository_impl.dart
```

## 10. DOMAIN LAYER DESIGN

The domain layer contains pure business logic and is independent of UI or data sources.

### Entities:

Entities represent important business objects. Examples:

- Note
- Vehicle
- Transaction
- Reminder
- Document
- Receipt
- Recipe

**Rule:** Entities should not depend on UI.

### Repository Interfaces:

Repository contracts belong to the domain layer:

```
abstract class NoteRepository {  
    Future<List<Note>> getNotes();  
    Future<void> createNote(Note note);  
    Future<void> updateNote(Note note);  
    Future<void> deleteNote(String id);  
}
```

The domain layer should not care whether implementation uses SQLite, Drift, Isar, Files, or API.

### Use Cases:

Use cases represent meaningful business operations. Examples:

- create\_note.dart
- update\_note.dart
- delete\_note.dart
- search\_notes.dart
- calculate\_total.dart
- schedule\_reminder.dart

**Rule:** Do not create meaningless use cases for every line of code. Create use case when it represents meaningful business operation or improves separation/testability.

## 11. PRESENTATION LAYER DESIGN

The presentation layer contains UI and UI state management.

### Presentation Layer Structure:

```
presentation/  
  providers/  
    feature_state_provider.dart  
    feature_controller.dart  
  screens/  
    home_screen.dart  
    create_screen.dart  
    detail_screen.dart  
  widgets/  
    feature_item_widget.dart  
    shared_widgets.dart
```

### Providers:

Riverpod providers manage state. They should call use cases, not access repositories directly.

### Screens & Widgets:

- Screens represent full pages or major UI sections
- Widgets are reusable UI components
- Should be stateless where possible
- UI state comes from Riverpod providers

### UI State Handling:

Every screen must handle:

- Loading states
- Empty states
- Error states
- Success states

## 12. ROUTING & NAVIGATION

Use GoRouter for all navigation.

### Router Structure:

```
app/router/  
■■■ app_router.dart  
■■■ route_names.dart  
■■■ route_paths.dart
```

### Benefits:

- Type-safe navigation
- Deep linking support
- Clear route definitions
- Separation of navigation logic from UI

## 13. STATE MANAGEMENT (RIVERPOD)

**Use Riverpod for all state management.**

### **Key Riverpod Concepts:**

```
final notesProvider = FutureProvider((ref) async {  
  return await ref.watch(noteRepositoryProvider).getNotes();  
});  
  
final noteControllerProvider = StateNotifierProvider(  
  (ref) => NoteController(ref.watch(noteRepositoryProvider))  
);
```

### **Rules:**

- Providers should be immutable
- Inject dependencies through providers
- Use FutureProvider for async data
- Use StateNotifierProvider for mutable state
- UI consumes providers via ConsumerWidget or useProvider

## 14. DATABASE & PERSISTENCE

### Choose appropriate local persistence based on needs:

**Isar:** Recommended. Fast, typed, local database. No backend.

**Drift:** SQL-based. More control, more setup.

**SQLite:** Basic SQL. Good for complex queries.

**SharedPreferences:** Simple key-value. Small data only.

**Files:** JSON/text files. Simple use cases.

### Database Design Rules:

- Define clear schema
- Use migrations for schema changes
- Index frequently queried fields
- Normalize data where appropriate
- Test data persistence across app restart

## 15. ERROR HANDLING

### Proper error handling is mandatory:

- **Data Validation Errors:** Invalid input from user
- **Database Errors:** Data access failures
- **Network Errors:** Future API connectivity issues
- **State Errors:** Invalid app state transitions
- **Runtime Errors:** Unexpected exceptions

### Error Strategy:

- Use custom exception classes in domain layer
- Catch and map exceptions at repository layer
- Show user-friendly error messages
- Log errors for debugging
- Implement fallback/retry logic where appropriate



## 16. CODE QUALITY & NAMING

### Naming Conventions:

**Classes:** PascalCase: NoteModel, VehicleRepository

**Functions/Methods:** camelCase: createNote(), updateVehicle()

**Variables:** camelCase: userNote, totalExpense

**Constants:** camelCase: defaultTimeout, maxRetries

**Files:** snake\_case: note\_model.dart, vehicle\_repository.dart

**Providers:** descriptive\_provider: notesProvider, vehicleControllerProvider

**Enums:** PascalCase values: status.completed, priority.high

### File Organization:

- Keep files < 500 lines
- One main class per file
- Use meaningful filenames
- Group related logic together

### Avoid:

- X update
- X changes
- X final
- X final2
- X new
- X test
- X data
- X temp

### Never Commit:

- Secrets • API keys • Passwords • Private credentials

## 17. AI DEVELOPMENT PROCESS (13 PHASES)

### MANDATORY SEQUENCE - DO NOT SKIP PHASES

**Phase 1: Requirements Analysis:** Before coding: Explain what app does, who uses it, what problem it solves, V1 scope, important assumptions.

**Phase 2: Feature Breakdown:** Identify actual features. Do not create artificial features.

**Phase 3: User Flows:** Define major user journeys (e.g., Open → Create → Save → View).

**Phase 4: Screen Architecture:** List screens, navigation, dialogs, bottom sheets, important UI states.

**Phase 5: Data Model:** Identify entities, fields, relationships, IDs, validation, derived values.

**Phase 6: Database Design:** Design tables, columns, relationships, indexes, queries, migrations.

**Phase 7: Architecture Design:** Map project into Presentation/Domain/Data layers. Define repositories, data sources, use cases.

**Phase 8: Folder Structure:** Generate PROJECT-SPECIFIC folder structure based on actual requirements.

**Phase 9: Dependencies:** Choose only necessary packages. For each: explain package, purpose, why needed.

**Phase 10: Foundation:** Implement routing, theme, DI container, database, error handling, core utilities.

**Phase 11: Feature Implementation:** Build ONE feature at a time: Domain → Data → Presentation → Testing.

**Phase 12: Validation:** After each feature: Run analyzer, check compilation, run tests, check UI, test persistence, verify edge cases.

**Phase 13: Final Review:** Review architecture, data layer, UI states, code quality, security, testing.

**CRITICAL:** Provide detailed output before implementation for each phase. Do not generate large code blocks without analysis.

## 18. DO NOT OVER-ENGINEER

Clean Architecture does NOT mean creating hundreds of files or unnecessary abstractions.

### Do NOT create:

- 5 classes for a simple boolean
- 10 services for a simple feature
- Repositories that do nothing
- Use cases that only forward one trivial call
- Unnecessary abstractions, interfaces, services
- Backend infrastructure for hypothetical V2

Architecture should match the complexity of the application.

### The GOAL is:

- ✓ Maintainable
- ✓ Testable
- ✓ Understandable
- ✓ Scalable

### NOT:

- ✗ Maximum number of files

## 19. FUTURE SCALABILITY

V1 should be simple. The architecture should be scalable.

### Target:

```
Simple V1
+
Clean Architecture
+
Clear Boundaries
+
Good Data Model
=
Easy Future Expansion
```

### Future Backend Addition:

If backend is added later, architecture should support it:

```
V1:
Presentation → Domain → Local Repository → Local Database

Future:
Presentation → Domain → (Repository Interface)
                        ■          ■
                    Local Repository Remote Repository
                        ↓          ↓
                    Local Database API/Server
```

The presentation layer should not need complete rewriting.

**Do NOT implement future backend unless explicitly requested.**

## 20. FINAL ARCHITECTURE CHECKLIST

**Before declaring the project complete, verify all these items:**

- Flutter/Dart project ✓
- Feature-first architecture ✓
- Clean Architecture ✓
- Repository pattern ✓
- Riverpod state management ✓
- GoRouter navigation ✓
- Local persistence where required ✓
- No unnecessary backend ✓
- No server database ✓
- No admin panel ✓
- No paid APIs ✓
- UI separated from data ✓
- Business logic separated ✓
- Loading states handled ✓
- Empty states handled ✓
- Error states handled ✓
- Input validation ✓
- Responsive UI ✓
- Theme system ✓
- Reusable components ✓
- Important tests ✓
- README documentation ✓
- Architecture documentation ✓
- No hardcoded secrets ✓
- No unnecessary dependencies ✓
- No unnecessary abstractions ✓
- Future scalability considered ✓

## FINAL AI DIRECTIVE

Treat this document as the MANDATORY engineering standard.

Treat the developer's project description as the source of truth for functionality.

Do NOT invent unnecessary features.

Do NOT simplify architecture into a quick demo.

Do NOT blindly copy example folder structure.

Do NOT create unnecessary complexity.

**ANALYZE THE PROJECT FIRST.**

**DESIGN ARCHITECTURE BASED ON ACTUAL REQUIREMENTS.**

**THEN IMPLEMENT FEATURE-BY-FEATURE.**

The final code must be:

- ✓ **Production-quality**
- ✓ **Maintainable**
- ✓ **Testable**
- ✓ **Offline-first where applicable**
- ✓ **Properly structured**
- ✓ **Easy for another developer to understand**
- ✓ **Ready for future expansion**

The architecture must support future growth  
without forcing a complete rewrite.